

Terrain-Nerfacto Training: Opacity-Threshold Height Supervision

Terrain-Nerf / NEMo

May 21, 2026

1 Purpose and Concept

Terrain-Nerfacto extends Nerfacto with a jointly trained NEMo terrain height field

$$z = h(x, y).$$

The NeRF density field learns volumetric scene geometry from images, while the NEMo height field learns a compact single-valued terrain surface in raw dataset coordinates. The current default method configuration in `terrain_nerf/config.py` uses:

```
height_supervision_mode = "threshold"
height_supervision_rays = "vertical"
height_field_type = "smooth-grid"
```

Threshold supervision uses NeRF opacity as a self-supervised geometric signal. It renders rays through the current NeRF density, finds the first sampled ray depth where cumulative opacity crosses a fixed threshold, converts that hit point back into raw dataset coordinates, and trains NEMo so that $h(x, y)$ matches the hit height.

This makes the height field learn from NeRF/ray evidence rather than direct high-resolution GT DEM supervision. The DEM point cloud is still loaded for height-field bounds, output normalization, optional anchors, metrics, and optional NeRF/DEM coupling losses.

2 Training Components

The implementation is centered on these files in `/home/addai/NeRF/terrain-nerf`:

Concept	Implementation
Method defaults	<code>terrain_nerf/config.py</code>
Pipeline wiring	<code>terrain_nerf/pipeline.py</code>
Raw/world coordinate transforms	<code>terrain_nerf/datamanager.py</code>
Nerfacto + NEMo model	<code>terrain_nerf/model.py</code>
Threshold depth extraction	<code>_threshold_depth</code>
Height supervision targets	<code>_depth_supervision_targets</code>
Height loss	<code>get_loss_dict</code> threshold branch
Optional density mask	<code>_height_density_mask</code>
Vertical NeRF depth diagnostics	<code>scripts/render_vertical_nerf_depth.py</code>

3 Coordinate Frames

Terrain-Nerfacto has to keep two coordinate systems aligned:

- **Raw dataset/world frame:** DEM point cloud and NEMo height field live here.
- **Nerfstudio training frame:** camera rays, NeRF sampling, and Nerfacto density live here.

The datamanager builds transforms from `transforms.json`, Nerfstudio dataparser transform, and dataparser scale:

$$\mathbf{x}_{nerf} = T_{raw \rightarrow nerf}(\mathbf{x}_{raw}), \quad \mathbf{x}_{raw} = T_{nerf \rightarrow raw}(\mathbf{x}_{nerf}).$$

The pipeline loads `dem_pc.npy`, gives the model both transforms, and initializes NEMo bounds from the DEM point cloud:

$$x \in [x_{\min}, x_{\max}], \quad y \in [y_{\min}, y_{\max}].$$

The height field therefore predicts raw-world elevation, even when the supervision target comes from a NeRF ray sampled in Nerfstudio coordinates.

4 Height Field Initialization

When `set_dem_point_cloud` runs, the model:

1. filters finite XYZ DEM points;
2. sets height-field XY bounds from DEM min/max;
3. builds either a `SmoothGridHeightField` or `ResidualMLPHeightField`;
4. configures output normalization from a subsample of DEM heights;
5. optionally builds a coarse DEM anchor grid;
6. optionally caches a regular DEM grid for ray/DEM depth losses.

In threshold mode, the DEM point cloud is not the main per-step height target. It supplies the domain and normalization needed for stable NEMo learning.

5 Standard NeRF Rendering Step

For each training batch, Terrain-Nerfacto follows the usual Nerfacto rendering path:

1. receive a camera ray bundle;
2. apply camera optimization if training;
3. sample points along rays with the proposal sampler;
4. evaluate field density and color;
5. convert density to alpha weights;
6. render RGB, depth, expected depth, and accumulation;
7. compute standard Nerfacto losses from the parent class.

For ray sample i , Nerfacto-style volumetric rendering uses density-derived opacity weights w_i . These are the per-sample contributions used by RGB/depth rendering and by threshold height supervision.

6 Opacity-Threshold Supervision

6.1 Ray Choice

Threshold supervision can use either camera rays or vertical rays:

$$\text{height_supervision_rays} \in \{\text{camera}, \text{vertical}\}.$$

With **camera**, the same camera rays used for image training also produce height targets. With **vertical**, the model samples random raw-world (x, y) locations inside the height-field bounds, starts above the terrain, converts the ray into Nerfstudio coordinates, and points it downward in raw vertical direction.

For vertical rays:

$$\begin{aligned} \mathbf{o}_{raw} &= [x, y, z_{\max} + 20], \\ \mathbf{d}_{nerf} &= \frac{T_{raw \rightarrow nerf}([x, y, z_{\max} + 19]) - T_{raw \rightarrow nerf}([x, y, z_{\max} + 20])}{\|T_{raw \rightarrow nerf}([x, y, z_{\max} + 19]) - T_{raw \rightarrow nerf}([x, y, z_{\max} + 20])\|_2}. \end{aligned}$$

This matters because a camera ray can hit terrain obliquely or be influenced by visibility, while a vertical ray asks the NeRF density field for a top-down terrain height signal at a sampled XY point.

6.2 Accumulated Opacity

For each supervision ray, the proposal sampler generates ray samples with midpoint depths

$$t_i = \frac{t_i^{\text{start}} + t_i^{\text{end}}}{2}.$$

The density field produces volumetric weights w_i . The cumulative opacity is:

$$C_i = \sum_{j=1}^i w_j.$$

Given an absolute opacity threshold τ , configured by **threshold_supervision_value**, the threshold hit index is:

$$k = \min\{i : C_i \geq \tau\}.$$

The threshold depth is:

$$t_\tau = t_k.$$

The implementation uses the first sample whose cumulative sum crosses τ :

```
cdf = torch.cumsum(weights, dim=-2)
hit = cdf >= threshold
first_hit = hit.to(dtype=torch.int64).argmax(dim=-2, keepdim=True)
t_depth = sample_depths.gather(dim=-2, index=first_hit).squeeze(-2)
```

6.3 Validity Gate

A threshold target is accepted only when the ray has enough total accumulation:

$$A = C_{\text{last}}.$$

The effective minimum accumulation is:

$$A_{\min} = \max(\text{threshold_supervision_min_accumulation}, \tau).$$

The target is valid if:

$$A \geq A_{\min},$$

the target point is finite, and the raw-world XY point lies inside the NEMo height-field bounds.

7 Target Point Construction

The threshold depth is converted to a Nerfstudio point along the supervision ray:

$$\mathbf{p}_{nerf} = \mathbf{o}_{nerf} + t_{\tau} \mathbf{d}_{nerf}.$$

Then the model maps it back to raw dataset coordinates:

$$\mathbf{p}_{raw} = T_{nerf \rightarrow raw}(\mathbf{p}_{nerf}) = [x_{\tau}, y_{\tau}, z_{\tau}].$$

The NEMo training target is the raw-world height:

$$h(x_{\tau}, y_{\tau}) \rightarrow z_{\tau}.$$

The model stores these targets in `outputs["height_depth_points_raw"]` and the validity mask in `outputs["height_depth_valid"]`.

8 Height Loss

For valid threshold targets, the height field produces normalized training predictions:

$$\hat{z}_{norm} = h_{train}(x_{\tau}, y_{\tau}).$$

The target height is also converted into the height field’s training target space:

$$z_{norm} = \text{target}_{train}(x_{\tau}, y_{\tau}, z_{\tau}).$$

Threshold mode uses Huber loss:

$$\mathcal{L}_{height} = \text{Huber}_{\delta_{norm}}(\hat{z}_{norm} - z_{norm}).$$

The configured Huber delta is specified in raw-world vertical units:

$$\delta_{raw} = \text{threshold_supervision_huber_delta}.$$

The implementation converts it into normalized height-field units:

$$\delta_{norm} = \frac{\delta_{raw}}{\text{height_field.output_scale}}.$$

The final loss term added to the Nerfacto losses is:

$$\mathcal{L}_{total} = \mathcal{L}_{nerfacto} + \lambda_h \mathcal{L}_{height} + \mathcal{L}_{optional},$$

where

$$\lambda_h = \text{height_loss_mult}.$$

9 Why Threshold Instead of Expected Depth

Expected depth averages over all density along a ray:

$$t_{expected} = \frac{\sum_i w_i t_i}{\sum_i w_i}.$$

That can be biased by broad density clouds, floaters, or multiple surfaces. Weighted median and quantile statistics reduce some averaging artifacts, but still choose a relative fraction of the ray’s total accumulation:

$$C_i \geq qA.$$

Threshold supervision instead uses an absolute opacity crossing:

$$C_i \geq \tau.$$

For terrain, this tends to represent the first confident surface contact along the ray. In vertical-ray mode, it is especially aligned with the desired product: a top-down terrain height field.

10 Masking Options

10.1 Image and Dataset Masks

The datamanager remains Nerfstudio-compatible and can use image masks from the dataset/parser stack. These masks influence which camera rays contribute to standard image training. They are separate from the learned height-field density mask described below.

10.2 Density Height Mask

Terrain-Nerfacto can optionally couple the NeRF density field to the learned height field during forward rendering:

```
enable_density_height_mask = True
height_mask_margin = ...
height_mask_softness = ...
```

For every NeRF sample, the model converts the sample position to raw coordinates and predicts the NEMo height at the sample XY:

$$\hat{z} = h(x, y).$$

It defines clearance:

$$c = \hat{z} + m - z_{sample},$$

where $m = \text{height_mask_margin}$.

With a hard mask:

$$M = \begin{cases} 1, & c \geq 0, \\ 0, & c < 0. \end{cases}$$

With a soft mask:

$$M = \sigma\left(\frac{c}{s}\right),$$

where $s = \text{height_mask_softness}$.

The density is multiplied by this mask:

$$\sigma'(\mathbf{x}) = M(\mathbf{x}) \sigma(\mathbf{x}).$$

Interpretation: samples above the learned terrain surface can be suppressed, with optional margin and softness. This is a strong coupling from NEMo back into NeRF density, so it should be used carefully. If NEMo is biased early in training, the mask can remove density needed by NeRF to recover.

11 Coupling Options

Terrain-Nerfacto supports multiple levels of coupling between NeRF, NEMo, and DEM data:

- **One-way NeRF to NEMo:** `threshold/depth` supervision trains NEMo from rendered NeRF geometry. This is the default threshold path.
- **NEMo to NeRF density masking:** `enable_density_height_mask` uses the learned height field to suppress density above the terrain.
- **DEM to NEMo direct supervision:** `height_supervision_mode="dem"` trains NEMo from sampled DEM points.
- **DEM anchor with depth mode:** `depth` mode can add a coarse DEM anchor using `depth_supervision_dem_anchor_mpp` and `depth_supervision_dem_anchor_loss_mult`.
- **DEM to NeRF depth loss:** `nerf_dem_depth_loss_mult > 0` supervises NeRF expected depth against ray/DEM intersections.
- **Height regularization:** `height_tv_loss_mult` and `height_curvature_loss_mult` smooth the NEMo surface.

For experiments that must learn from NeRF/ray evidence alone, the direct high-resolution DEM losses should be treated as optional anchors or diagnostics rather than the primary training signal.

12 Training Step Summary

One threshold-supervised training step can be summarized as:

1. Sample a batch of camera rays for standard Nerfacto image training.
2. Render RGB, depth, expected depth, and accumulation through the current density field.
3. If threshold height supervision is active, choose camera rays or generate vertical rays.
4. Render density weights along the height-supervision rays.
5. Compute cumulative opacity $C_i = \sum_{j \leq i} w_j$.
6. Select the first depth t_τ where $C_i \geq \tau$.
7. Convert $\mathbf{o} + t_\tau \mathbf{d}$ from Nerfstudio coordinates to raw-world coordinates.
8. Keep only finite, in-bounds, sufficiently accumulated targets.
9. Train NEMo with Huber loss against the valid raw-world hit heights.
10. Add optional TV/curvature, DEM, NeRF/DEM, or density-mask coupling terms depending on config.

13 Important Configuration Knobs

Option	Meaning
<code>height_supervision_mode</code>	<code>threshold</code> , <code>depth</code> , <code>quantile</code> , or <code>dem</code>
<code>height_supervision_rays</code>	Use <code>vertical</code> or <code>camera</code> rays for depth/threshold targets
<code>threshold_supervision_value</code>	Absolute opacity threshold τ
<code>threshold_supervision_min_accumulation</code>	Minimum ray opacity required for valid targets
<code>threshold_supervision_huber_delta</code>	Raw-world Huber delta for height loss
<code>height_loss_mult</code>	Weight on NEMo height supervision loss
<code>enable_density_height_mask</code>	Enable NEMo-to-NeRF density suppression
<code>height_mask_margin</code>	Allow density up to this margin above NEMo
<code>height_mask_softness</code>	Soft sigmoid transition width for density mask
<code>nerf_dem_depth_loss_mult</code>	Optional direct DEM supervision on NeRF expected depth
<code>height_tv_loss_mult</code>	Optional NEMo total-variation smoothing
<code>height_curvature_loss_mult</code>	Optional NEMo curvature/Laplacian smoothing

14 Diagnostics

Useful checks are:

- `height_depth_valid_fraction`: whether threshold rays are producing enough valid targets.
- final `height_field_surface.html`: learned NEMo surface and optional GT DEM error heatmap.
- vertical NeRF depth render: separates NeRF density geometry from NEMo height-field output.
- GT DEM metrics: RMSE, MAE, max absolute error, bias, and 2D error heatmap when DEM is available.

The key diagnostic distinction is that NeRF vertical depth and NEMo height are separate signals. Threshold supervision transfers a surface estimate from NeRF density into NEMo, but density geometry can still be offset, broad, or artifact-prone while the learned height field is smoother or biased differently.

15 Practical Interpretation

Threshold-supervised Terrain-Nerfacto is a two-surface training system. The NeRF density field is the volumetric image-fitting model. The NEMo height field is a compact terrain prior trained from ray-derived surface hits. Opacity thresholding defines what counts as the first reliable NeRF surface. Vertical rays make that signal top-down and terrain-like. Masking and DEM losses can make the systems more tightly coupled, but they also increase the risk that one biased component constrains the other too early.